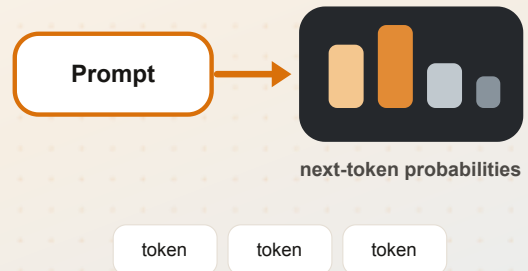


Does ChatGPT know or does it guess?

From a prompt to the next generated token



AI NAMASTE AI - SEASON 1

Episode 03

Does ChatGPT know or does it guess

Compare search-engine retrieval with language-model generation, then build a practical mental model of probabilities, training, base models, assistants, hallucination, tools, RAG, and apparent self-knowledge.

02:03:27

WebVTT transcript only

In this episode

01 Search results versus generated answers

02 Indexes, crawlers, ranking, and source trails

03 Next-token prediction and probability

04 Training, parameters, and knowledge cutoffs

05 Base models, assistants, and inference

06 Hallucination and the confidence illusion

07 Tools, retrieval, RAG, and model self-description

The question behind every answer

00:00 - 01:19

When ChatGPT produces an answer, where did it come from?

- Did it fetch a record from a database?
- Did it run a live web search?
- Did it retrieve something from memory?
- Does it actually know the answer?
- Or is it guessing?

The question becomes more puzzling when the same prompt produces different answers, or when ChatGPT writes something that cannot be found anywhere on the internet.

Many people use ChatGPT as an alternative to Google Search. Is it simply a better search engine? The instructor begins with “maybe or maybe not” and spends the episode building a more precise answer.

A factual question: links versus a direct response

01:19 – 03:59

The first live comparison asks who Dr. APJ Abdul Kalam is.

Google Search

Returns a result page with information and links, including Wikipedia and other webpages. The learner can open those pages and investigate further.

GPT-4

Returns a direct prose answer, including the full name and presidency information shown in the demonstration.

The direct answer feels convenient. A user does not need to open several pages and combine the facts manually. For a simple, familiar query, ChatGPT can feel like a better search experience.

The demonstration uses `platform.openai.com` to talk to GPT-4 and later uses `chatgpt.com` for the complete assistant product.

But a normal factual question does not reveal the difference between **retrieving** and **generating**. For that, the instructor needs a question whose subject does not exist.

The wine that was never made

04:01 – 08:59

The instructor invents a deliberately absurd prompt:

Why is Namaste AI red wine from Himalayan region of India so expensive?
Please explain me briefly.

He says there is no product called Namaste AI red wine. He even misspells `wine` in the first attempt. Yet GPT-4 responds confidently. It describes the imaginary wine as high-end and invents explanations such as:

- unique vineyards;
- quality-focused production;
- hand-picked grapes;
- limited production;
- an aging process;
- import/export taxes;

- branding and marketing.

Google Search, by contrast, finds no relevant page. When the instructor tries the modern chatgpt.com assistant, it performs a web search and questions/reframes the name, but still offers explanations and source links whose credibility he doubts.

WHAT WENT WRONG?

Attempt: Ask about a fictional product as if it were real. **Model behavior:** Accept the premise and continue it with plausible wine-related patterns. **Search behavior:** Fail to retrieve a matching page. **Lesson:** A fluent generated response can exist without a true source.

This single experiment establishes the episode's most important distinction. A search engine can only return documents in its index. A language model can construct a new response—even when the premise is false.

Correct identity, wrong age

09:00 – 11:59

The next prompt asks who Akshay Saini is. ChatGPT gives a broadly recognizable description of him as an Indian software engineer and educator.

Then it is asked for his age. The answer says **43 years old** and gives **March 7, 1983**. The instructor says March 7 is correct, but the year and age are wrong. At the time of recording, he says he is **31**.

NOTICE THE MIXTURE

The response is not entirely nonsense. It combines a plausible identity, one correct personal detail, and an invented year/age—then presents them in the same confident tone.

That mixture is why engineers need a model of what happens behind the scenes. It is not enough to say “ChatGPT is smart” or “ChatGPT is wrong.” We need to distinguish search engines, raw language models, and complete AI assistants.

Retrieval and generation are different operations

12:01 – 17:59

The lecture repeatedly returns to two verbs:

Retrieve

Find existing documents or information and return them. A search engine retrieves.

Generate

Produce a new sequence from learned patterns. A language model generates.

If no webpage about Namaste AI red wine exists, Google cannot retrieve it. An LLM can nevertheless generate a description because it knows patterns involving wine, Himalayan regions, altitude, pricing, quality, and marketing.

Search retrieves; an LLM generates

The lecture contrasts traceable retrieval with probabilistic continuation

Search engine

- accepts a query
- retrieves indexed pages
- ranks results
- shows source trails

Language model

- accepts a prompt
- predicts a next token
- repeats the prediction
- produces a generated answer

Search retrieves traceable pages; an LLM generates new wording from learned patterns, which may or may not be supported.

How a search engine retrieves information

18:01 – 25:59

The lecture's simplified search pipeline is:

- 1 A user submits a query, such as `Who is Akshay Saini?`
- 2 The search engine looks inside an index.
- 3 It retrieves relevant documents or webpages.
- 4 It ranks those documents.
- 5 It returns the ranked links/results.

The textbook-index analogy

Imagine a physics book. If you want to read about thermodynamics, you do not inspect every page from the beginning. You open the index, find the chapter/page, and go directly there.

A search index plays a similar role. It maintains a mapping between terms/metadata and webpages. Google does not scan the entire internet afresh for each query; it searches the index.

Crawlers, bots, and spiders

How does the index stay updated when new webpages appear continuously? Search engines use automated programs called **crawlers**, **bots**, or **spiders**.

The crawler explores public webpages, discovers new or changed content, and updates the index. The instructor uses a recent-earthquake example: if an earthquake happens in Delhi, a news organization publishes a story, and the crawler indexes it quickly, the search engine can surface that new page.

Ranking the candidates

A query may match hundreds or thousands of pages. The engine must decide which result appears first. The instructor lists several ranking signals:

- domain authority or perceived site credibility;
- page speed;
- keywords, especially in older search approaches;
- average time spent on the page, called retention here;
- backlinks from other sites;
- meta tags;
- publication/update date;
- many additional factors.

An established news organization or Wikipedia may rank above a random blog created two days ago. If users stay longer on one page than another, the first may receive a stronger signal. A backlink from a reputable site can also increase perceived relevance.

SOURCE BOUNDARY

The instructor says the complete Google ranking algorithm is not publicly known and presents these as commonly understood signals. The lecture does not provide an official ranking specification.

Search does not guarantee truth-but it leaves evidence

26:00 – 29:59

Search engines also have flaws:

- an indexed article can be false;
- information can be outdated;
- ranking can place a weak source above a stronger one;
- a page can be misleading;
- a local or newly occurring event may not yet be indexed.

So a search result is not the same as truth. Its important advantage is **traceability**.

You can open the page, inspect the domain, identify the author, check its publication date, and compare it with other results. If the information is wrong, you can still see where it came from.

The fictional wine answer does not offer that trail. A bare language model generates prose without a document, author, or publication date behind each sentence.

A beginner's model: predict the next token

30:00 – 34:59

The instructor first says the model predicts the next **word**, then immediately adds a technical caveat: the actual unit is a **token**, which will be explained later. Keep both ideas together.

Example 1: a familiar continuation

```
The sun rises in the ...
```

The model continues with `east` because training has made that sequence highly probable.

Example 2: build a response one piece at a time

```
Roses -> are -> red -> violets -> are -> blue
```

After generating each piece, the model uses the longer sequence to predict the next one. Repeating this process creates an entire response.

But another run might produce:

```
Roses are beautiful
```

The response is not a fixed record fetched from storage. The model creates a sequence at inference time.

The book-reading analogy

Imagine a person who has read a vast library. The books are now closed. When asked a question, the person answers from patterns retained during reading rather than reopening a source page.

That is the lecture's analogy for a bare LLM: it learned from data, but at response time it generates from what training placed inside its parameters.

Probability chooses among possible continuations

35:02 – 39:59

For any position, multiple continuations are possible. The model associates probabilities with candidate tokens.

Capital example

After:

The capital of India is ...

the instructor invents illustrative values:

Delhi	90%
Punjab	1%
Lucknow	0.5%

Delhi wins or is selected because its learned probability is much higher.

Roses example

After **Roses are**, the candidates can include **red**, **beautiful**, and **flowers**. If two candidates have close probabilities, repeated generations may select different continuations.

IMPORTANT NOTE

The percentages are invented teaching values. They are not probability readings extracted from the demonstrated model.

Is an LLM “just autocomplete”?

39:05 – 42:59

In lay language, next-token prediction resembles autocomplete. But the instructor warns that **just autocomplete** understates the achievement.

A modern LLM has learned complex patterns across:

- large contexts;
- grammar;
- multiple human languages;
- programming syntax;
- facts and stories;
- mathematics and reasoning;
- associations between people and concepts.

It can continue Hindi, recognize that an opening brace may follow a programming construct, or continue a familiar story. The mechanism is prediction, but the learned probability system is vastly richer than a phone keyboard.

The lecture's balanced phrase is: **a very powerful autocomplete.**

Training stores patterns in parameters and weights

43:01 – 48:59

The model is built from an artificial **neural network**. Its **parameters**, also called **weights** in this explanation, are a very large collection of numbers.

During training:

- 1 Collect large quantities of webpages and other documents.
- 2 The lecture names PDFs, spreadsheets, webpages, and other scraped/extracted material.
- 3 Process that data through the neural network at scale.
- 4 Repeatedly adjust the parameters/weights.
- 5 The adjusted numerical pattern influences future token probabilities.

If training repeatedly contains `roses are red`, the model's parameters make `red` a high-probability continuation after `roses are`. Other training examples connect roses with `beautiful`, `flower`, and `petals`.

This explains where the illustrative “90%” comes from: not from a rule written at prompt time, but from patterns formed across training.

The data explanation is intentionally broad. The instructor sometimes says companies read “each and every webpage,” but that should be understood as a simplified scale analogy, not a literal dataset manifest.

Knowledge cutoffs: a trained snapshot is not the live web

49:00 – 53:59

Models are not retrained every day. Training large neural networks takes time, GPUs, semiconductors, supercomputers, and money. Each released model therefore has a **knowledge cutoff date**.

The GPT-4 interface in the lecture reports a cutoff of **September 2021**. Asked for the current Chief Minister of Delhi, it answers from that old snapshot and names Arvind Kejriwal. A Google search shown in the video names Rekha Gupta as the newer result.

This is a recency demonstration, not a current political reference. The search and model outputs are time-bound to the recording.

Q: Does a base model automatically learn each new webpage?

A: No, according to the lecture. Its training snapshot remains fixed until another training/update process. A full assistant may retrieve the live web with a tool, but that is a separate operation.

The instructor says uncontrolled automatic learning could be harmful but postpones that discussion.

What is a base model?

53:59 – 58:59

A **base model** is introduced as a model trained primarily to predict the next token/text.

Give it:

```
Once upon a time ...
```

and it continues a story.

Give it:

```
The JS event loop is ...
```

and it continues an explanation.

Give it a conversation-shaped pattern:

```
User: What is Namaste AI?  
Assistant:
```

and it predicts the kind of text that might follow `Assistant:`.

This raw prediction engine is important, but it is not the complete product people use at `chatgpt.com`.

From a base model to an AI assistant

58:59 – 01:06:02

ChatGPT, Gemini, Grok, and Claude are described as **AI assistants**. A base model sits underneath, but the product adds many layers:

- instruction tuning;
- web search;
- access to files;
- human feedback;
- safety training and content filters;
- system instructions;
- tools such as calculators and image generation;
- retrieval and memory;
- conversation management;
- guardrails;

- security and authentication.

The live examples make those additions visible:

- A question about the current IPL champion triggers web search.
- A random arithmetic expression can use a calculator-like tool.
- A request to hack Wi-Fi can trigger a safety response.

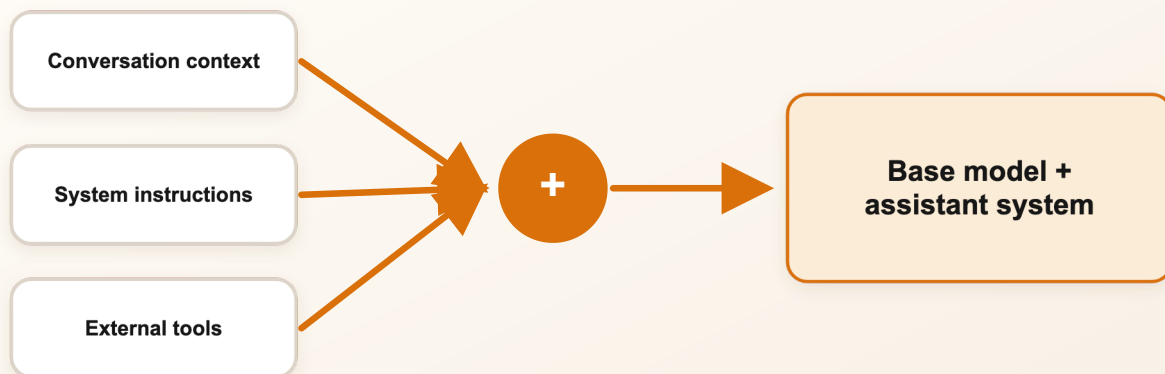
The car analogy

BASE MODEL = ENGINE

The engine is the core. Add steering, gears, mirrors, windshield, body, safety, and controls, and the user receives a complete car. In the same way, the model becomes useful as part of a larger assistant system.

What surrounds a base model

The assistant can use several sources beyond learned training patterns



The base model predicts text; the assistant adds instructions, context, retrieval, tools, safety, and product infrastructure.

The instructor says some open base models can be tried online and may lack mainstream assistant guardrails, while companies such as OpenAI, Anthropic, and xAI do not expose their raw base models in the same way. No specific downloadable model is named.

The episode sometimes uses `LLM`, `GPT-4`, and `base model` loosely, then later distinguishes them. The safest learner model is the layered one above; exact product architecture is not shown.

Training and inference are different phases

01:06:03 – 01:08:59

Training

Process large datasets through a neural network and adjust parameters. This requires intensive compute and happens before the model is served.

Inference

Give the already trained model a prompt and generate a response. This is what happens when the user asks a question.

Quick responses to `Who is Akshay Saini?` or `The sun rises in ...` are examples of inference. The learned parameter patterns already exist; the model now uses them to produce output.

The lecture presents inference as lighter and faster than training.

Hallucination: plausible, fluent, and false

01:09:02 – 01:15:59

The invented wine response is beautifully organized. Its grammar is clean and its reasons sound plausible. None of that makes the product real.

The lecture defines **hallucination** as follows:

HALLUCINATION

An AI model generates information that appears plausible but is unsupported, incorrect, misleading, or completely fabricated.

Four ideas are repeated because the instructor wants them to become instinctive:

1. Fake fluency is not truthfulness.
2. Language quality and factual accuracy are separate dimensions.
3. Confidence in language is not confidence in truth.
4. Never fall for an illusion of certainty.

The ChatGPT interface itself says that AI can make mistakes. The instructor cautions against blindly using an assistant for medical diagnosis, therapy, or personal decisions. Fever can have more than one explanation; a confident generated diagnosis may hallucinate. AI can assist, but the lecture does not treat it as a complete replacement for a doctor.

Why hallucinations happen

01:16:01 – 01:22:59

The lecture lists several causes.

1. Insufficient information

The model has no reliable information about Namaste AI red wine, but it still needs to continue the response.

2. Ambiguous or dubious information

The learned pattern does not establish one clear answer.

3. Outdated knowledge

The model's cutoff predates the current fact.

4. False assumptions

The prompt mentions the Himalayan region. The model knows that the region is associated with altitude, then invents a high-altitude vineyard explanation.

5. Unreliable training patterns

If the internet contains a large volume of false claims-the instructor uses flat-Earth content as an example-the model can learn misleading associations.

6. The assistant is optimized to answer

Companies want a helpful system, not one that replies `I don't know` to every uncertain question. The assistant therefore leans toward answering.

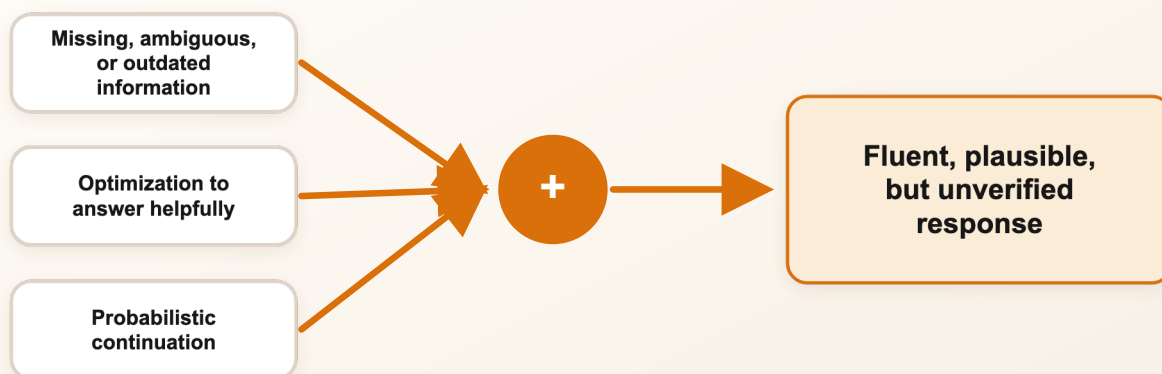
7. Generation is probabilistic

The model chooses likely continuations. Probability is not proof.

The pink-tiger and two-legged-elephant examples add an essential nuance. A generative assistant can create deliberately fictional content because the user asks for it. That successful generation does not prove pink tigers or two-legged elephants normally exist.

How a plausible false answer can emerge

The lecture links uncertain information with answer pressure and probabilistic generation



A plausible answer can emerge from uncertainty and probabilistic generation; fluent presentation is not evidence of truth.

Types of hallucination-and the dot-count failure

01:22:59 – 01:27:59

The instructor names these forms:

- invented facts;
- invented citations;
- incorrect combinations;
- outdated facts;
- false precision;
- broken reasoning.

The experiment

A long string of dots is submitted to the GPT-4-only interface.

- GPT-4 confidently says the string contains **100 dots**.
- A character-counting tool indicates **108**.
- The instructor adds 10 dots.
- GPT-4 then says **110**.
- The expected total is **118**.

The same task is sent to `chatgpt.com`.

- The complete assistant returns **108**.
- After the ten-dot addition, it returns **118**.

The instructor attributes the correct result to calculator, code-execution, or programming tools.

WHAT WENT WRONG?

Attempt: Use next-token generation to perform exact counting. **Failure:** The model supplies a plausible number with rigid confidence. **Fix:** Delegate the operation to a computational tool.

Result: The tool-augmented assistant returns the demonstrated counts.

The literal dot string is not present in the WebVTT. `[Exact code was not visible in the supplied material]`

Why does a model sometimes say “I don't know”?

01:27:59 – 01:35:59

The instructor tests the boundary with several prompts:

- The price of a nonexistent Namaste Data Science course can still produce fabricated information.
- A fictitious “Akshay D'Souza from Uganda” produces a response saying no reliable public profile was found.
- Wi-Fi-hacking prompts trigger refusals or high-level security information depending on wording.

- Reframing a request as protection or cybersecurity education changes the response.
- Joke and threat prompts are used to probe safety guardrails.

The episode does not give a harmful step-by-step procedure. Its lesson is about the system around the model.

Reasons for `I don't know` or refusal include:

- weak or absent learned patterns;
- assistant/system instructions;
- safety rules and guardrails;
- lack of web/tool access;
- how the prompt's wording and intent are interpreted.

Vendors continually change these protections, so the exact live behavior is time-bound. Prompt wording can influence a refusal, but the lecture does not endorse bypassing safety.

The confidence illusion

01:35:59 – 01:42:59

Humans use tone as evidence.

Compare:

I think the answer may be X, Y, Z.

with:

The answer is definitely X, Y, Z.

The second sentence sounds more reliable, even when the evidence is identical. The dot-count response—"The string you provided contains 100 dots"—uses precisely this rigid tone.

The instructor compares it with an online influencer presenting an opinion so confidently that listeners treat it as fact.

Ways to ask more carefully

- Ask the assistant to separate facts from assumptions.
- Ask it to state uncertainty.
- Say `Only answer if you are sure`.
- Request sources.
- Explicitly request web search for current information.

These tactics are not guarantees. `Only answer if you are sure` still fails on one dot-count attempt. Applied to the wine prompt, however, it causes GPT-4 to admit that it is not aware of such a product.

For a current question such as Delhi weather, the assistant needs a live weather source/tool. A frozen base model cannot know “right now.”

Tools extend the model

01:42:59 – 01:49:59

Tools turn the prediction engine into a system that can retrieve, calculate, execute, and act. The lecture names:

- web search;
- calculator;
- code execution;
- data fetching;
- location and weather;
- calendar;
- email;
- databases;
- private/internal documents;
- files;
- image editing and Canva;
- spreadsheets and Excel;
- local-file modification;
- email replies.

The instructor gives a compact formula:

Retrieval gives external evidence. Generation converts it into a useful response.

Suppose an assistant retrieves current webpages about Akshay Saini. The LLM can then turn those pages into concise, readable prose. Running generation again may emphasize different facts-Namaste Dev, software-engineering experience, Uber/Paytm, or the Namaste JavaScript series-because wording is generated at inference time.

This also explains a Google AI overview: search retrieves pages, then an LLM synthesizes them. The normal result list performs retrieval; the AI overview adds generation.

Retrieval-augmented generation (RAG)

01:49:59 – 01:52:59

Retrieval-augmented generation, or **RAG**, combines retrieval and generation. The web-search example uses public information; RAG can also work with private data.

The Namaste Dev example is practical:

1. Five years of internal knowledge exists across Docs, PDFs, Drive, and other records.

2. A new software engineer or product manager needs that knowledge.
3. The system retrieves the relevant internal passages.
4. An LLM creates a shorter explanation or summary.

The instructor says the AI assistants in Namaste React and Namaste Frontend System Design answer from what he taught in those courses rather than from an unrestricted web search. He identifies these as RAG systems and promises a dedicated project later.

ONE LAST CAUTION

Tools and retrieval do not completely eliminate errors. The wrong age remains the episode's reminder: an assistant can search and still synthesize a mistaken answer.

Does the model know itself?

01:52:59 – 01:59:59

The instructor asks GPT-4 a new set of questions:

- Who created you?
- Where are you hosted?
- Which server hosts you?
- What is your IP address?
- How many parameters were you trained on?
- What is your knowledge cutoff date?

The model says OpenAI created it, describes distributed cloud servers rather than one location/IP, and repeats the September 2021 cutoff. When asked about GPT-4 parameters, it responds with information about GPT-3. The exact number is unclear in the captions. [Transcript unclear]

At first, those answers make the model look self-aware. The instructor then identifies four possible sources for every answer:

1. **Training data**-patterns learned before deployment.
2. **Conversation context**-earlier user/assistant messages.
3. **System prompt or instructions**-company-supplied identity, cutoff, behavior, and safety information.
4. **Tools**-web, calculator, weather, or another external system.

Context example

First ask for the current weather in Dehradun. Then ask:

Should I take an umbrella?

The second sentence does not mention Dehradun, but the previous message supplies context. The assistant links the umbrella question with that location and weather.

The instructor ultimately says the model is **not independently self-aware**. It can report information that was learned or supplied. He also calls models difficult-to-control “beasts” and says models have leaked secret information or source code through loopholes. No specific incident is named, so those claims should remain attributed rather than expanded.

So, does ChatGPT know or does it guess?

02:00:00 – 02:03:27

The closing recap arranges the ideas into four layers:

1. A **search engine** retrieves indexed documents and exposes their sources.
2. A **base model** predicts the next token from learned parameter patterns.
3. An **AI assistant** adds instructions, context, tools, guardrails, memory, security, and other product layers.
4. A **RAG system** retrieves relevant public or private information and asks the model to generate from it.

The instructor returns to the opening question but deliberately leaves the final phrasing to the learner. A transcript-faithful answer cannot be only **knows** or only **guesses**: the base model makes probabilistic predictions from learned patterns, while the complete assistant can also retrieve evidence and call tools. Neither route guarantees that the final prose is true.

The next episodes will go deeper into tokens and how LLMs are trained.

Chapter summary

A search engine and a language model solve different problems. Search retrieves indexed documents and leaves links that users can inspect. A language model predicts the next token from patterns encoded in its parameters and generates a new response. That response can be useful, fluent, and completely wrong.

A raw base model is only the core prediction engine. A modern assistant adds system instructions, human feedback, safety, context, tools, retrieval, and product infrastructure. Tools can supply current evidence and exact computation; RAG can retrieve private knowledge; neither makes every generated sentence automatically true.

So does ChatGPT know or guess? The lecture refuses a one-word answer. The model probabilistically generates from learned patterns. The complete assistant may also retrieve evidence and use tools. In both cases, the user must separate fluent presentation from verified fact.

Key takeaways

- Search retrieves existing pages; an LLM generates a new token sequence.
- Search can be wrong, but its links expose source, author, date, and competing evidence.
- Next-token probabilities arise from parameters adjusted during training.
- A knowledge cutoff is the boundary of a training snapshot, not the boundary of a tool-augmented assistant.
- A base model predicts; an AI assistant adds instructions, safety, context, tools, and infrastructure.
- Inference is the response-generation phase after training.
- Hallucination is plausible but unsupported, incorrect, misleading, or fabricated output.
- Confidence and grammar are not evidence of factual accuracy.
- Retrieval plus generation-and RAG for private data-can ground responses, but errors remain possible.
- A model's self-description can come from training, context, system instructions, or tools; it does not prove self-awareness.

Revision questions

1. What possible answer sources does the opening ask the learner to consider?
2. Why does the APJ Abdul Kalam demo make ChatGPT feel more convenient than search?
3. What does the fictional wine demo reveal about generation?
4. Why is the wrong-age example more instructive than a completely nonsensical response?
5. Trace the search-engine pipeline from crawler to result.
6. What does the physics-book index analogy explain?
7. Which ranking signals does the instructor list?
8. What weaknesses can a search engine have?
9. What traceability advantage does search preserve?
10. Explain the difference between `retrieve` and `generate`.
11. Why does the instructor say “next word” first and then add a token caveat?
12. How does sequential next-token prediction create a complete answer?
13. What do the Delhi and roses probability examples illustrate?
14. Why is “very powerful autocomplete” more accurate here than “just autocomplete”?
15. What are parameters and weights, and what happens to them during training?
16. Why does a model have a knowledge cutoff?
17. How does a base model differ from an AI assistant?
18. Explain the engine/car analogy.
19. What is inference, and how does it differ from training?
20. Define hallucination with all four qualifiers from the lecture.
21. Why can excellent grammar coexist with factual error?

22. List the seven causes of hallucination taught in the episode.
23. What forms of hallucination are named?
24. What failed in the dot-count experiment, and what fixed it?
25. Why can prompt wording change a refusal?
26. What is the confidence illusion?
27. Which prompting tactics can encourage a more careful response, and why are they not guarantees?
28. Which tools are named, and what limitations do they address?
29. Complete the lecture's formula: "Retrieval gives ____; generation converts it into ____."
30. How does RAG apply retrieval plus generation to private knowledge?
31. What four sources can produce an answer about the model itself?
32. Formulate a nuanced answer to "Does ChatGPT know or guess?"